

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»**

*ИНСТИТУТ Информационных компьютерных наук (ИКН)
КАФЕДРА Автоматизированных систем управления (АСУ)
НАПРАВЛЕНИЕ 09.03.00 ГРУППА БИВТ-24-6*

КУРСОВОЙ ПРОЕКТ

*ПО КУРСУ Клиент-серверные приложения.
ТЕМА: Спорт менеджер*

Студент Рыжков Л.С.



*Проверил:
Абросимов Н. А.
Шелудяков П. А.
КТН, СИС, доцент
Микитенко И.И.*

Оценка выполнения курсового проекта _____

Москва 2026 г.

СОДЕРЖАНИЕ

1. Предметная область.....	4
2. Постановка задачи.....	5
2.1. Функциональные требования	5
2.2. Нефункциональные требования.....	6
3. Описание архитектуры.....	7
3.1. Клиентская часть	7
3.2. Серверная часть	7
3.3. Технологический стек	8
3.4. Взаимодействие компонентов.....	8
4. Описание структуры БД	10
4.1. Таблицы.....	10
4.2. Представления	11
4.3. Функции и хранимые процедуры.....	12
4.4. Триггеры.....	12
5. Описание серверной части	13
5.1. Конфигурация (пакет config).....	13
5.2. Пакет database	14
5.3. Слой моделей (пакет models).....	14
5.4. Слой репозитория (пакет repository).....	14
5.4.1. UserRepository	15
5.4.2. SportRepository	15
5.4.3. PlayerRepository	15
5.5. Слой сервисов (пакет service).....	15
5.5.1. AuthService	16
5.5.2. SportService, TeamService, MatchService	16
5.6. Промежуточное ПО (пакет middleware).....	16
5.7. Слой обработчиков (пакет handler).....	17
6. Описание клиентской части	19
6.1. Структура проекта.....	19
6.2. Модуль api.js — HTTP-клиент	19
6.3. Компонент App.jsx — управление состоянием	20
6.4. Экраны и пользовательские сценарии.....	20
6.4.1. Экран аутентификации	20
6.4.2. Главный экран — Блок «Профиль»	21
6.4.3. Главный экран — Форма добавления/редактирования	21
6.4.4. Главный экран — Таблица дисциплин.....	22
Заключение.....	23

Список литературы.....	25
------------------------	----

1. Предметная область

Спорт является одной из важнейших сфер общественной жизни, охватывающей сотни миллионов людей по всему миру как в роли участников, так и в роли болельщиков. Спортивные организации различного уровня — от любительских клубов до профессиональных лиг — ежедневно сталкиваются с необходимостью эффективно управлять большим объёмом структурированной информации: данными об игроках, командах, видах спорта и соревнованиях.

В отсутствие специализированного инструментария менеджеры и тренеры вынуждены прибегать к ручному ведению записей — в бумажных журналах, таблицах Microsoft Excel или разрозненных текстовых файлах. Такой подход несёт в себе целый ряд системных проблем.

Во-первых, проблема разрозненности и дублирования данных. Когда информация о командах хранится в одном файле, об игроках — в другом, а о матчах — в третьем, неизбежно возникают рассинхронизация и дублирование. Изменение названия команды потребует ручной правки во всех связанных документах, и любая ошибка может привести к неконсистентному состоянию данных.

Во-вторых, проблема разграничения доступа. При многопользовательской работе (несколько тренеров в одной организации) крайне сложно обеспечить, чтобы каждый видел и мог редактировать только свои записи, не влияя на данные коллег.

В-третьих, проблема обеспечения целостности. Без автоматических проверок возможно создание логически некорректных записей: матч между командой и ею же самой, отрицательный счёт, привязка игрока к несуществующей команде и т.д.

В-четвёртых, проблема потери истории изменений. При удалении записи об игроке вся связанная с ним история (участие в матчах, статистика) теряется безвозвратно.

Разработка специализированного клиент-серверного веб-приложения позволяет системно решить все перечисленные проблемы: централизовать хранение в надёжной реляционной СУБД, обеспечить разграничение доступа через механизм аутентификации, автоматически поддерживать целостность данных на уровне базы данных с помощью ограничений и триггеров, а также вести архив удалённых записей.

2. Постановка задачи

Целью курсовой работы является разработка клиент-серверного веб-приложения «Спортивный Менеджер» для учёта спортивных данных пользователя с использованием технологий Go, React и PostgreSQL.

2.1. Функциональные требования

Разрабатываемое приложение должно обеспечивать следующие функциональные возможности:

Управление учётными записями пользователей:

- Регистрация нового пользователя с указанием имени и пароля. Пароль должен храниться в хешированном виде (bcrypt).
- Аутентификация по имени и паролю с выдачей JWT-токена, действующего 24 часа.
- Обновление имени пользователя в профиле.
- Удаление собственного аккаунта вместе со всеми связанными данными (каскадное удаление).

Управление видами спорта (CRUD):

- Создание записи о виде спорта с указанием названия и типа (например, «Футбол» / «Мини»).
- Просмотр полного списка своих видов спорта.
- Редактирование названия и типа существующей записи.
- Удаление вида спорта (каскадно удаляет связанные команды и их игроков).
- Получение количества команд, связанных с видом спорта (через хранимую функцию БД).

Управление командами:

- Создание команды с указанием названия, города и вида спорта.
- Просмотр списка своих команд.

Управление игроками:

- Добавление игрока через хранимую процедуру БД (имя, фамилия, дата рождения, позиция, команда).

- Перевод игрока в другую команду через хранимую процедуру.
- Удаление игрока через хранимую процедуру (с автоматическим архивированием триггером).
- Просмотр профилей игроков через представление БД (с данными о команде и виде спорта).

Управление матчами:

- Создание матча между двумя разными командами с указанием даты и счёта.
- Автоматическая проверка на уровне БД: команды должны быть разными, счёт не может быть отрицательным.

2.2. Нефункциональные требования

- Изоляция данных: каждый пользователь видит и редактирует только свои записи; запросы к БД всегда фильтруются по `user_id`.
- Безопасность: пароли хранятся только в хешированном виде; все защищённые эндпоинты требуют валидного JWT-токена.
- Контейнеризация БД: PostgreSQL запускается через Docker Compose для простоты развёртывания.
- Разделение фронтенда и бэкенда: взаимодействие осуществляется исключительно через REST API с JSON.

3. Описание архитектуры

Приложение построено по классической трёхзвенной клиент-серверной архитектуре. Пользователь взаимодействует с браузерным клиентом (React SPA), который посредством HTTP-запросов с JSON-телом обращается к серверному REST API (Go). Сервер, в свою очередь, выполняет запросы к реляционной СУБД PostgreSQL 15.

3.1. Клиентская часть

Клиентская часть реализована как одностраничное приложение (SPA — Single Page Application) на базе библиотеки React 18. Сборка проекта осуществляется инструментом Vite, обеспечивающим быструю горячую перезагрузку в режиме разработки и оптимизированный бандл для продакшена. Клиент запускается на порту 5173 (dev-режим) и взаимодействует с сервером через fetch API, передавая JWT-токен в заголовке Authorization.

3.2. Серверная часть

Серверная часть написана на языке Go и использует исключительно стандартную библиотеку net/http для маршрутизации HTTP-запросов. Это обеспечивает минимальную зависимость от сторонних фреймворков и высокую производительность. Сервер запускается на порту 8080.

Серверная часть строго следует принципу разделения ответственности и организована в следующие слои:

- Слой конфигурации (config) — загружает параметры подключения к БД и порт сервера из .env файла посредством библиотеки godotenv. Если .env не найден, используются системные переменные окружения.
- Слой базы данных (database) — инициализирует пул соединений *sql.DB с PostgreSQL через драйвер lib/pq.
- Слой моделей (models) — определяет Go-структуры, соответствующие сущностям предметной области: User, Sport, Team, Player, Match, PlayerProfileView.
- Слой репозитория (repository) — реализует паттерн Repository. Каждый репозиторий отвечает за взаимодействие с одной сущностью БД и инкапсулирует все SQL-запросы.

- Слой сервисов (service) — содержит бизнес-логику. Сервисы вызывают методы репозитория, обеспечивают хеширование паролей и формирование JWT.
- Слой обработчиков (handler) — HTTP-контроллеры. Принимают запросы, декодируют JSON, вызывают сервис и отправляют ответ.
- Слой промежуточного ПО (middleware) — JWT-аутентификация. Извлекает и верифицирует токен, передаёт user_id в контекст запроса для последующей изоляции данных.

3.3. Технологический стек

Таблица 1 — Технологический стек проекта

Компонент	Технология / Библиотека	Версия	Роль в проекте
Серверная часть	Go (Golang)	1.22+	Основной язык бэкенда
HTTP-маршрутизатор	net/http (stdlib)	—	Мультиплексирование запросов
Аутентификация	golang-jwt/jwt	v5.3.1	Генерация и проверка JWT
Хеширование паролей	golang.org/x/crypto/bcrypt	v0.52.0	Безопасное хранение паролей
Драйвер БД	github.com/lib/pq	v1.12.3	Подключение к PostgreSQL
Конфигурация	github.com/joho/godotenv	v1.5.1	Загрузка .env файла
СУБД	PostgreSQL	15 Alpine	Реляционная база данных
Контейнеризация	Docker Compose	—	Развёртывание PostgreSQL
Клиентская часть	React	18	UI-фреймворк (SPA)
Сборщик	Vite	—	Сборка и dev-сервер фронтенда
Иконки	lucide-react	—	SVG-иконки интерфейса

3.4. Взаимодействие компонентов

Порядок обработки типичного запроса (на примере создания вида спорта):

- Пользователь вводит данные в форму на клиенте и нажимает «+ Добавить».
- api.js формирует POST-запрос к /sports с JSON-телом { name, type } и заголовком Authorization: Bearer <token>.
- Middleware AuthMiddleware перехватывает запрос, парсит JWT, извлекает user_id и помещает его в контекст.
- SportHandler.Create декодирует тело запроса в структуру models.Sport и вызывает SportService.CreateSport.
- SportService.CreateSport делегирует вызов SportRepository.Create.
- SportRepository.Create выполняет INSERT INTO sports ... RETURNING id и возвращает созданный объект.
- Handler возвращает клиенту HTTP 201 Created с JSON-телом созданного ресурса.
- Клиент обновляет список дисциплин вызовом api.getSports().

4. Описание структуры БД

База данных `sports_db` развёртывается в СУБД PostgreSQL 15 (контейнер Docker). Структура БД полностью определяется в файле `init.sql`, который автоматически выполняется при первом запуске контейнера. Схема включает шесть таблиц, одну служебную таблицу-архив, три представления, три функции, три хранимые процедуры и три триггера.

4.1. Таблицы

Ниже приведена вербальная модель предметной области с описанием каждой таблицы, её столбцов и межтабличных связей.

Таблица 2 — Структура таблиц базы данных

Таблица	Поля (столбцы)	Ограничения и связи
users	id (PK, SERIAL), username (VARCHAR, UNIQUE), password_hash (VARCHAR)	Все остальные таблицы ссылаются через user_id
sports	id (PK), name (VARCHAR), type (VARCHAR), user_id (FK)	user_id → users(id) ON DELETE CASCADE
teams	id (PK), name (VARCHAR), city (VARCHAR), sport_id (FK), user_id (FK)	sport_id → sports(id) CASCADE, user_id → users(id) CASCADE
players	id (PK), first_name, last_name, birth_date (DATE), team_id (FK, NULL), position, user_id (FK)	team_id → teams(id) ON DELETE SET NULL, user_id CASCADE
matches	id (PK), home_team_id (FK), away_team_id (FK), match_date (TIMESTAMP), home_score, away_score, user_id (FK)	home/away_team_id → teams(id) CASCADE, user_id CASCADE
player_archive	id, first_name, last_name, deleted_at (TIMESTAMP), user_id	Заполняется триггером, внешних ключей нет

Таблица `users` является центральной для всей схемы. Она хранит учётные записи пользователей. Поле `username` объявлено с ограничением `UNIQUE` — система не позволит зарегистрировать двух пользователей с одинаковым именем. Поле `password_hash` содержит результат `bcrypt`-хеширования пароля — оригинальный пароль нигде не хранится.

Таблицы `sports`, `teams`, `players` и `matches` содержат поле `user_id` с внешним ключом на `users(id)` `ON DELETE CASCADE`. Это гарантирует, что при удалении пользователя все его данные автоматически удаляются без дополнительного кода на уровне приложения.

В таблице `players` поле `team_id` использует стратегию `ON DELETE SET NULL`, а не `CASCADE`. Это означает, что при удалении команды игроки не удаляются, а становятся «свободными агентами» (`team_id = NULL`), что соответствует реальной спортивной практике.

Таблица `player_archive` не имеет внешних ключей и используется исключительно для хранения истории удалённых игроков. Она заполняется автоматически триггером `tg_archive_player` после каждого удаления из таблицы `players`.

4.2. Представления

Таблица 3 — Представления базы данных

Представление	SQL-источники	Назначение
<code>vw_player_profiles</code>	<code>players LEFT JOIN teams LEFT JOIN sports</code>	Профили игроков: имя, позиция, команда, вид спорта
<code>vw_match_results</code>	<code>matches JOIN teams t1, JOIN teams t2</code>	Результаты матчей с именами обеих команд и счётом
<code>vw_team_roster_count</code>	<code>teams LEFT JOIN players + COUNT + GROUP BY</code>	Количество игроков в каждой команде пользователя

Представление `vw_player_profiles` является ключевым для клиентского интерфейса. Оно соединяет данные трёх таблиц и позволяет получить полную информацию об игроке — включая название его команды и вида спорта — единственным запросом. Использование `LEFT JOIN` вместо `INNER JOIN` гарантирует, что «свободные агенты» (`team_id IS NULL`) также попадают в результат.

Представление `vw_match_results` решает проблему самосоединения: таблица `teams` соединяется с `matches` дважды (под псевдонимами `t1` и `t2`) для получения имён домашней и гостевой команд в одной строке результата.

Представление `vw_team_roster_count` использует агрегатную функцию `COUNT` и `GROUP BY` для подсчёта игроков в каждой команде. Применение `LEFT JOIN` гарантирует, что команды без игроков отображаются с нулевым счётчиком.

4.3. Функции и хранимые процедуры

Таблица 4 — Функции и процедуры базы данных

Объект БД	Тип	Описание
fn_calculate_age(bdate DATE)	Функция	Вычисляет возраст в годах: EXTRACT(YEAR FROM AGE(bdate))
fn_get_total_matches(t_id, u_id)	Функция	Считает количество матчей команды (домашних + гостевых)
fn_get_sport_team_count(s_id, u_id)	Функция	Возвращает число команд, привязанных к виду спорта
sp_insert_player(...)	Процедура	Добавляет игрока в таблицу players
sp_update_player_team(p_id, new_t_id, u_id)	Процедура	Переводит игрока в другую команду
sp_delete_player(p_id, u_id)	Процедура	Удаляет игрока текущего пользователя

Все функции и процедуры написаны на языке PL/pgSQL. Функции `fn_calculate_age`, `fn_get_total_matches` и `fn_get_sport_team_count` используются для получения вычисляемых данных без необходимости усложнять SQL-запросы в коде приложения.

Хранимые процедуры `sp_insert_player`, `sp_update_player_team` и `sp_delete_player` инкапсулируют операции модификации данных. `PlayerRepository` вызывает их через конструкцию `CALL`, что обеспечивает единую точку входа для изменения записей игроков и упрощает возможные будущие доработки (добавление валидации, аудита и т.д.).

4.4. Триггеры

Таблица 5 — Триггеры базы данных

Триггер	Событие	Действие
tg_check_teams	BEFORE INSERT OR UPDATE ON matches	Проверяет <code>home_team_id</code> $\neq$ <code>away_team_id</code> . При нарушении — <code>RAISE EXCEPTION</code>
tg_check_scores	BEFORE INSERT OR UPDATE ON matches	Проверяет <code>home_score</code> ≥ 0 AND <code>away_score</code> ≥ 0 . Отрицательный счёт — ошибка
tg_archive_player	AFTER DELETE ON players	Копирует данные удалённого игрока в <code>player_archive</code> с меткой времени

Триггер `tg_check_teams` выполняется перед каждой вставкой или обновлением записи в таблице `matches`. Он обращается к значениям `NEW.home_team_id` и `NEW.away_team_id` и при их совпадении вызывает `RAISE EXCEPTION`, немедленно откатывая транзакцию. Это исключает возможность создания матча «команда против себя» — ошибки, которая была бы допустима на уровне приложения без данной проверки.

Триггер `tg_check_scores` аналогичным образом защищает от записи отрицательного счёта матча.

Триггер `tg_archive_player` выполняется после удаления строки из таблицы `players`. Он читает данные удалённой строки через псевдозапись `OLD` и вставляет их в `player_archive` вместе с текущей временной меткой (`NOW()`). Таким образом, история игроков сохраняется даже после их удаления из основной таблицы.

Ниже приведён пример кода триггерной функции архивирования:

Листинг 1 – пример кода триггерной функции архивирования

```
CREATE FUNCTION fn_tg_archive_player()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO player_archive (id, first_name, last_name, deleted_at,
user_id)
VALUES (OLD.id, OLD.first_name, OLD.last_name, NOW(), OLD.user_id);
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_archive_player
AFTER DELETE ON players
FOR EACH ROW EXECUTE FUNCTION fn_tg_archive_player();
```

5. Описание серверной части

Серверная часть приложения разработана на языке программирования Go (Golang) версии 1.22. Выбор Go обусловлен высокой производительностью, встроенной поддержкой конкурентности через горуты и каналы, а также богатой стандартной библиотекой, достаточной для построения REST API без необходимости использования тяжёлых фреймворков.

5.1. Конфигурация (пакет config)

Пакет `config` реализует загрузку конфигурации из переменных окружения. Библиотека `godotenv` читает файл `.env` и экспортирует переменные в окружение процесса. Далее `os.Getenv` извлекает значения. Строка подключения к PostgreSQL собирается из отдельных переменных `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` и передаётся в `database.NewPostgresDB`. Если файл `.env` отсутствует, система использует уже установленные системные переменные — это обеспечивает совместимость с облачными средами развёртывания.

5.2. Пакет `database`

Пакет `database` отвечает за инициализацию соединения с PostgreSQL. Функция `NewPostgresDB` принимает строку подключения, открывает пул соединений через `sql.Open` с драйвером `lib/pq` и немедленно проверяет доступность СУБД вызовом `db.Ping()`. При ошибке приложение завершается с соответствующим сообщением. Пул соединений `*sql.DB` является потокобезопасным и автоматически управляет переиспользованием соединений.

5.3. Слой моделей (пакет `models`)

Пакет `models` содержит Go-структуры, представляющие сущности предметной области. Теги `json` управляют сериализацией/десериализацией JSON:

- `User`: поле `Password` имеет тег `json:"password,omitempty"` — оно заполняется при входящих запросах (регистрация, логин), но не включается в исходящие ответы. Поле `PasswordHash` имеет тег `json:"-"` и никогда не сериализуется.
- `Player`: поле `TeamID` объявлено как `*int` (указатель), что позволяет хранить `NULL` из базы данных — игрок может не быть привязан ни к какой команде.
- `PlayerProfileView`: отдельная структура для данных из представления `vw_player_profiles` — содержит денормализованные поля `TeamName` и `SportName`, упрощая клиентский код.

5.4. Слой репозитория (пакет `repository`)

Каждый репозиторий принимает `*sql.DB` в конструкторе и использует его для выполнения запросов. Рассмотрим ключевые особенности реализации каждого репозитория.

5.4.1. UserRepository

Метод `FindByUsername` использует `QueryRow` и `Scan` для поиска пользователя. Метод `CreateUser` выполняет `INSERT` с возвратом `id` через `RETURNING id` и записывает его в структуру через `Scan`. Метод `DeleteUser` выполняет прямой `DELETE` — благодаря каскадным ограничениям БД все связанные записи удаляются автоматически.

5.4.2. SportRepository

Метод `GetAll` фильтрует записи по `user_id`, обеспечивая изоляцию данных. Метод `GetTeamCount` вызывает хранимую функцию `fn_get_sport_team_count` через `SELECT fn_get_sport_team_count($1, $2)` — стандартный способ вызова PostgreSQL-функций из Go.

5.4.3. PlayerRepository

Все три мутлирующих операции (`Create`, `UpdateTeam`, `Delete`) выполняются через `CALL`-вызовы хранимых процедур: `CALL sp_insert_player($1, ...)`, `CALL sp_update_player_team($1, $2, $3)`, `CALL sp_delete_player($1, $2)`. Метод `GetProfiles` использует представление `vw_player_profiles`, что позволяет получить обогащённые данные игроков без сложных `JOIN`-запросов в коде приложения.

Пример реализации метода `Create` репозитория игроков:

Листинг 2 – реализация метода `Create`

```
func (r *PlayerRepository) Create(p *models.Player, userID int) error {
    _, err := r.db.Exec(
        "CALL sp_insert_player($1, $2, $3, $4, $5, $6)",
        p.FirstName, p.LastName, p.BirthDate,
        p.TeamID, p.Position, userID,
    )
    return err
}
```

5.5. Слой сервисов (пакет `service`)

Сервисный слой инкапсулирует бизнес-логику и отделяет её от деталей транспортного уровня (HTTP) и уровня хранения (SQL).

5.5.1. AuthService

AuthService является наиболее сложным сервисом приложения. Метод Register получает объект User с открытым паролем, хеширует его функцией `bcrypt.GenerateFromPassword` с параметром `bcrypt.DefaultCost` (стоимость 10 — баланс между безопасностью и производительностью), после чего сохраняет пользователя через `UserRepository`. Метод Login выполняет обратную операцию: находит пользователя по имени, сравнивает хеш `bcrypt.CompareHashAndPassword`, и при успехе формирует JWT-токен с клеймами `user_id` и `exp` (время истечения через 24 часа), подписанный секретным ключом HS256.

Пример реализации метода Login:

Листинг 3 – реализация метода Login

```
func (s *AuthService) Login(username, password string) (string, error) {
    user, err := s.repo.FindByUsername(username)
    if err != nil {
        return "", errors.New("invalid credentials")
    }
    err = bcrypt.CompareHashAndPassword(
        []byte(user.Password), []byte(password),
    )
    if err != nil {
        return "", errors.New("invalid credentials")
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
        "user_id": user.ID,
        "exp":     time.Now().Add(time.Hour * 24).Unix(),
    })
    return token.SignedString(middleware.JwtSecret)
}
```

5.5.2. SportService, TeamService, MatchService

Остальные сервисы реализуют тонкий слой делегирования к соответствующим репозиториям. Их присутствие в архитектуре оправдано принципом единственной ответственности: при необходимости добавить валидацию (например, проверить уникальность имени команды или валидность даты матча) это можно сделать в сервисном слое, не трогая обработчики и репозитории.

5.6. Промежуточное ПО (пакет middleware)

Функция `AuthMiddleware` оборачивает основной мультиплексер и перехватывает все входящие запросы. Маршруты `/register` и `/login` освобождены от проверки токена — для них выполняется прямая передача управления далее. Для всех остальных маршрутов выполняется следующая последовательность:

- Извлечение заголовка Authorization. Отсутствие заголовка — 401 Unauthorized.
- Разбиение по пробелу и проверка формата «Bearer <token>». Несоответствие формату — 401 Unauthorized.
- Парсинг и верификация JWT через jwt.ParseWithClaims с тем же секретным ключом JwtSecret.
- Извлечение user_id из клеймов (приходит как float64 из JSON — конвертируется в int).
- Запись user_id в контекст запроса через context.WithValue. Последующие обработчики получают его через middleware.GetUserID(ctx).

5.7. Слой обработчиков (пакет handler)

Каждый обработчик соответствует одному HTTP-методу и пути. Метод RegisterRoutes регистрирует все маршруты обработчика в mux. Обработчики следуют единому паттерну: проверка метода → декодирование тела → вызов сервиса → формирование ответа.

Все API-маршруты приложения приведены в таблице ниже.

Таблица 6 — API-маршруты серверного приложения

Метод / Путь	Описание	Тело / Параметры	HTTP-ответ
POST /register	Регистрация нового пользователя	{ username, password }	201 + { token }
POST /login	Аутентификация пользователя	{ username, password }	200 + { token }
PUT /profile/username	Обновление имени пользователя	{ username }	200 OK
DELETE /profile	Удаление аккаунта и всех данных	—	200 OK
GET /sports	Список видов спорта пользователя	—	200 + JSON-массив
POST /sports	Создать новый вид спорта	{ name, type }	201 + { id, name, type }
PUT /sports	Обновить вид спорта	{ id, name, type }	200 OK
DELETE /sports?id=N	Удалить вид спорта по ID	?id=<id>	204 No Content

GET /sports/count?id=N	Число команд для вида спорта	?id=<id>	200 + { team_count }
GET /teams	Список команд пользователя	—	200 + JSON-массив
POST /teams	Создать команду	{ name, city, sport_id }	201 + { id, ... }
POST /players	Добавить игрока	{ first_name, last_name, birth_date, team_id, position }	201 Created
PUT /players/team	Перевести игрока в другую команду	{ id, team_id }	200 OK
DELETE /players?id=N	Удалить игрока (с архивацией)	?id=<id>	204 No Content
GET /players/profiles	Профили игроков (из представления)	—	200 + JSON-массив
POST /matches	Создать матч	{ home_team_id, away_team_id, match_date, home_score, away_score }	201 Created

Заметим, что маршруты /register и /login не требуют JWT-токена. Все остальные маршруты защищены промежуточным ПО AuthMiddleware и требуют заголовка Authorization: Bearer <token>. Сервер также реализует CORS-middleware, добавляющий заголовки Access-Control-Allow-Origin, позволяющие браузерному клиенту на порту 5173 обращаться к API на порту 8080.

6. Описание клиентской части

Клиентская часть приложения представляет собой одностраничное приложение (SPA — Single Page Application), разработанное на библиотеке React 18 с использованием сборщика Vite. Приложение обменивается данными с сервером исключительно через REST API, не обновляя страницу при навигации.

6.1. Структура проекта

Клиентский проект размещён в каталоге `Source/frontend/` и имеет следующую структуру:

- `index.html` — точка входа HTML-документа; содержит `div#root`, в который React монтирует приложение.
- `src/main.jsx` — инициализирует React и рендерит корневой компонент `<App />` в DOM-элемент `#root`.
- `src/App.jsx` — единственный компонент приложения, содержащий всю логику интерфейса и управления состоянием.
- `src/api.js` — HTTP-клиент, инкапсулирующий все запросы к серверному API.
- `src/App.css` / `src/index.css` — стилизация компонентов.
- `vite.config.js` — конфигурация сборщика.
- `package.json` — список зависимостей проекта.

6.2. Модуль `api.js` — HTTP-клиент

Модуль `api.js` экспортирует объект `api`, реализующий абстракцию над `fetch` API браузера. Центральным является метод `request(endpoint, method, body)`, который:

- Читает JWT-токен из `localStorage` по ключу `'token'`.
- Формирует объект `options` с нужным методом, заголовком `Content-Type: application/json` и — при наличии токена — `Authorization: Bearer <token>`.
- При наличии тела запроса сериализует его через `JSON.stringify` и добавляет в `options.body`.
- Выполняет `fetch` и при получении ответа с кодом `4xx/5xx` читает тело ошибки и выбрасывает исключение (`throw new Error`).

- При коде 204 (No Content) возвращает null, в остальных случаях — распарсенный JSON.

Остальные методы объекта `api` (`register`, `login`, `getSports`, `createSport`, `updateSport`, `deleteSport`, `updateUsername`, `deleteAccount`, `logout`) являются тонкими обёртками над `request` с фиксированными путями и методами. Метод `login` реализован отдельно — он также сохраняет полученный токен в `localStorage`.

6.3. Компонент `App.jsx` — управление состоянием

Компонент `App.jsx` является единственным React-компонентом приложения и управляет всем состоянием через хуки:

- `token` — JWT-токен пользователя. Инициализируется из `localStorage`. Наличие токена определяет, какой экран отображается.
- `isRegister` — флаг режима (вход / регистрация) на экране аутентификации.
- `username`, `password` — значения полей ввода на экране аутентификации.
- `error` — текст ошибки аутентификации для отображения пользователю.
- `sports` — массив видов спорта текущего пользователя.
- `name`, `type` — значения полей формы добавления/редактирования вида спорта.
- `editingId` — ID редактируемой записи (`null` — режим добавления, число — режим редактирования).
- `newUsername` — значение поля ввода нового имени пользователя в профиле.

Хук `useEffect` с зависимостью `[token]` вызывает `loadSports()` при первой загрузке компонента и при каждой смене токена — то есть после входа в систему список видов спорта загружается автоматически.

6.4. Экраны и пользовательские сценарии

Приложение отображает один из двух экранов в зависимости от наличия JWT-токена.

6.4.1. Экран аутентификации

При отсутствии токена пользователю отображается центрированная карточка с иконкой щита (`lucide-react Shield`), заголовком «Вход в систему» или «Регистрация»,

полями ввода имени и пароля, кнопкой отправки и ссылкой переключения режима. При ошибке (неверные учётные данные, занятое имя) отображается блок с текстом ошибки.

При успешном входе токен сохраняется в localStorage и в состояние token, что немедленно инициирует переключение на главный экран.

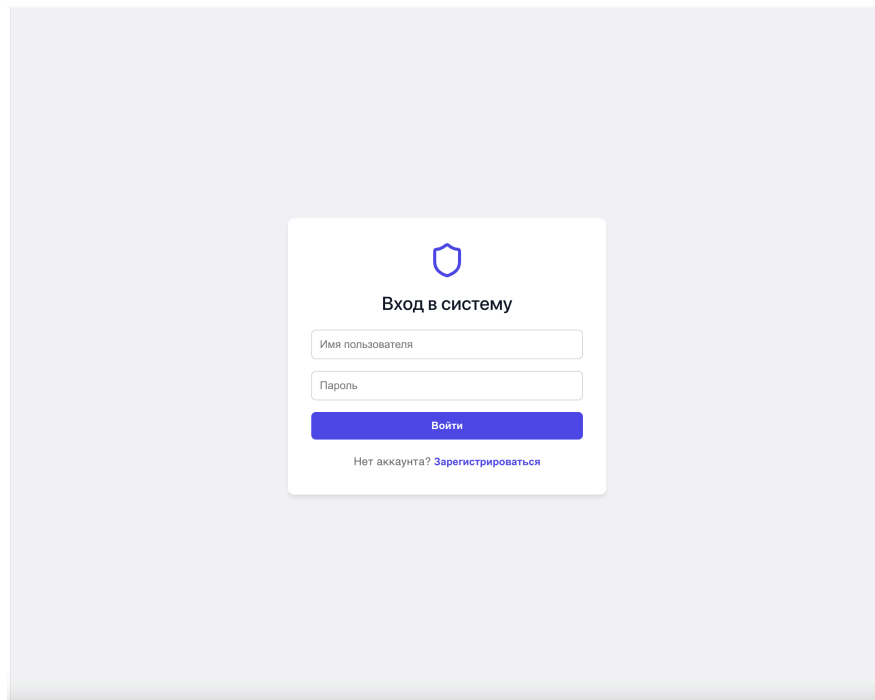


Рисунок 1 – экран авторизации

6.4.2. Главный экран — Блок «Профиль»

Верхний блок главного экрана содержит поле для ввода нового имени пользователя и кнопку «Изменить имя», вызывающую `api.updateUsername()`. Ниже расположена кнопка «Удалить аккаунт». При её нажатии браузер показывает нативный диалог подтверждения (Рисунок 4). При подтверждении вызывается `api.deleteAccount()`, токен удаляется из localStorage и пользователь возвращается на экран входа.

6.4.3. Главный экран — Форма добавления/редактирования

Блок «Добавить вид спорта» содержит два поля (Название, Тип) и кнопку «+ Добавить». При нажатии на кнопку «Редактировать» (иконка карандаша) в строке таблицы поля формы заполняются данными выбранной записи, а состояние `editingId` устанавливается в ID этой записи. При повторном нажатии кнопки в режиме

редактирования вместо создания выполняется обновление через `api.updateSport()`. После операции `editingId` сбрасывается в `null`.

Спортивный Менеджер Выйти

Профиль

Новое имя пользователя Изменить имя

Удалить аккаунт

Добавить вид спорта

Название Тип + Добавить

Ваши спортивные дисциплины

Другие пользователи не видят эти данные.

У вас пока нет записей.

Рисунок 2 – главный экран

6.4.4. Главный экран — Таблица дисциплин

Нижний блок содержит таблицу со столбцами ID, Название, Тип и Действия. В столбце Действия для каждой строки расположены две кнопки-иконки: карандаш (редактировать) и корзина (удалить). Нажатие на корзину показывает диалог подтверждения. После подтверждения вызывается `api.deleteSport(id)` и список обновляется.

Добавить вид спорта

Название Тип + Добавить

Ваши спортивные дисциплины

Другие пользователи не видят эти данные.

ID	Название	Тип	Действия
11	Футбол	Командный	
12	Бокс	Одиночный	
13	Хоккей	Командный	

Рисунок 3 – таблица добавленных дисциплин

Заключение

В ходе выполнения курсовой работы была рассмотрена предметная область управления спортивными данными. Выявлены ключевые проблемы, возникающие при отсутствии специализированного инструментария: разрозненность и дублирование данных, отсутствие разграничения доступа, риск нарушения целостности информации и невозможность отслеживания истории изменений.

В качестве решения разработано клиент-серверное веб-приложение «Спортивный Менеджер». Серверная часть реализована на языке Go и следует многослойной архитектуре Repository — Service — Handler, что обеспечивает чёткое разделение ответственности и высокую сопровождаемость кода. Клиентская часть выполнена в виде одностраничного приложения на React (Vite), взаимодействующего с сервером через REST API. Хранение данных обеспечивается СУБД PostgreSQL 15, развёртываемой посредством Docker Compose.

В рамках работы разработана и реализована схема базы данных, включающая шесть таблиц со связями через внешние ключи. Созданы три представления для упрощения клиентских запросов, три функции PL/pgSQL для расчёта агрегированных показателей, три хранимые процедуры для инкапсуляции операций с игроками и три триггера для автоматической проверки целостности данных и архивирования удалённых записей. Реализованы CRUD-операции для видов спорта и учётных записей пользователей, а также частичный CRUD для команд, игроков и матчей, доступные через задокументированный REST API.

Разграничение доступа обеспечивается JWT-аутентификацией: каждый запрос к защищённым эндпоинтам требует валидного токена, а все SQL-запросы фильтруются по user_id текущего пользователя, исключая возможность несанкционированного доступа к чужим данным.

Использованные инструменты: Go 1.22, React 18, Vite, PostgreSQL 15 Alpine, Docker Compose; библиотеки golang-jwt/jwt v5.3.1, golang.org/x/crypto/bcrypt, lib/pq v1.12.3, godotenv v1.5.1, lucide-react.

Результатом работы является функционирующее веб-приложение, демонстрирующее применение всех ключевых концепций дисциплины «Разработка

клиент-серверных приложений»: организации HTTP-сервера, взаимодействия с реляционной СУБД, механизмов безопасности на основе JWT и bcrypt, а также объектов базы данных — представлений, функций, процедур и триггеров.

Список литературы

- The Go Programming Language Specification / Google LLC. — URL: <https://go.dev/ref/spec> (дата обращения: 01.06.2026). [1]
- Документация стандартной библиотеки Go (net/http, database/sql, context) / Google LLC. — URL: <https://pkg.go.dev/std> (дата обращения: 01.06.2026). [2]
- PostgreSQL 15 Documentation / The PostgreSQL Global Development Group. — URL: <https://www.postgresql.org/docs/15/> (дата обращения: 01.06.2026). [3]
- Документация React 18 / Meta Platforms, Inc. — URL: <https://react.dev> (дата обращения: 01.06.2026). [4]
- Библиотека golang-jwt/jwt v5. — URL: <https://github.com/golang-jwt/jwt> (дата обращения: 01.06.2026). [5]
- Библиотека `golang.org/x/crypto/bcrypt`. — URL: <https://pkg.go.dev/golang.org/x/crypto/bcrypt> (дата обращения: 01.06.2026). [6]
- Документация Docker Compose / Docker, Inc. — URL: <https://docs.docker.com/compose/> (дата обращения: 01.06.2026). [7]
- ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2018. [8]
- Github / [Электронный ресурс] // : [сайт]. — URL: <https://github.com/balantrea/KR> (дата обращения: 01.06.2026).